

Local-Computation Optimization — Before / After Study

Five rounds, three skills, 36 A/B cells total, ~4.4M tokens spent measuring. Dated 2026-05-31.

This is a retrospective study of one specific optimization round — PRs #14 through #17 on [claude-skills](#) — which shipped two new skills (`skills-freshness` , `skills-quality`) that move work the LLM would otherwise do in-context into deterministic Python scripts. We measured the actual token delta with the model held constant, BEFORE arm = no SKILL.md (the pre-PR state for these new skills), AFTER arm = the merged SKILL.md. N=1 per cell, three models, two skills, six cells.

We are not claiming the principle of "minimize LLM tokens, maximize local computation" is broken. We are saying: on this sample of two skills, the win is real on Sonnet for freshness and on Haiku for quality, but on three of six cells the script dispatch costs more than it saves. The story is more interesting than the slogan.

HEADLINE NUMBERS (THIS ROUND ONLY)

2 of 28

Skills optimized this round

portfolio total = 14 global mikko-* + 14 Spacepotatis project skills



+2.5 %

Aggregate save across the 6 measured cells

629,795 tokens BEFORE → 613,918 tokens AFTER, summed across
2 skills × 3 models



3 of 6

Cells where AFTER cost *more* than BEFORE

Opus + Haiku on freshness, Sonnet on quality — all sign-flips, not noise



The +2.5% number is honest but headline-misleading on purpose. Half the cells lost ground; the headline averages winners and losers across only 6 measurements. Read the per-cell table below before forming an opinion about the optimization principle generally.

HOW TO READ THIS STUDY

- **Same-task, same-model A/B.** Each cell ran two fresh sub-agents on the same task — "audit the 14 mikko-* skills for [staleness | quality]" — differing only in whether they received the current SKILL.md. Same hour, same model, same target directory. So **the delta isolates the optimization** from background model drift and from the 1K–1M-token variance the registry's cross-session measurements absorb.
- **N=1 per cell.** Six cells, one trial each. Variance is real — the sign-flips below could move in a re-run. The aggregate number is fragile; the per-cell story is the load-bearing one.
- **BEFORE = no SKILL.md.** Both skills are new in this round, so "BEFORE" means the agent solved the task without any procedure. For future optimizations of pre-existing skills, BEFORE would be the SKILL.md at the parent of the optimizing commit.
- **Token accounting** = input + output + cache_creation_input_tokens, deduped by requestId per agent. cache_read excluded (tokens were paid upstream). Same convention as scripts/build-review-stats.mjs.
- **Token-only metric.** Maintenance cost of writing and shipping the Python script is not folded in. That cost is real and one-off; the per-use save (or loss) is the recurring side.
- **Rigor-exempt skills skipped.** audit, security-audit, modular-architecture-audit, save-roundtrip-audit, content-audit, equipment — their cost is the point. Including them would mix two different design goals.

SKILL	OPTIMIZING COMMITS	BEFORE TOKENS	AFTER TOKENS	SAVED	% SAVED	WHY THIS NUMBER
skills-freshness · claude-skills/skills/skills-freshness						
Sonnet (4-6) N=1, 2026-05-31	847154d · df58142	194,751 tokens	110,828 tokens	+83,923 tokens	+43%	Cold walk read 14 SKILL.mds + cross-checked refs in 28 turns; AFTER ran the Python script and reviewed the 5-row output in 38 short turns. The cache-creation savings on the cold-walk's massive prefix are what pays.
Opus (4-7) N=1, 2026-05-31	847154d · df58142	134,765 tokens	163,301 tokens	−28,536 tokens	−21%	Sign flip. Opus BEFORE terminated in 9 assistant turns at 135K — an unusually compact cold walk. AFTER (12 turns, 163K) pays for SKILL.md context + script dispatch + result review; on this model the cold walk was already cheap enough that the AFTER overhead exceeds the saving.
Haiku (4-5) N=1, 2026-05-31	847154d · df58142	64,192 tokens	108,806 tokens	−44,614 tokens	−70%	Largest sign flip in the study. Haiku BEFORE took 32 tiny assistant turns at 64K total — a cost-pinched walk of many small look-ups instead of fewer big reads. AFTER (11 turns, 109K) carries SKILL.md + script-output overhead that the cheap cold walk doesn't pay.

SKILL	OPTIMIZING COMMITS	BEFORE TOKENS	AFTER TOKENS	SAVED	% SAVED	WHY THIS NUMBER
skills-quality · claude-skills/skills/skills-quality						
Sonnet (4-6) N=1, 2026-05-31	778b40d · bb174dc	97,928 tokens	109,465 tokens	-11,537 tokens	-12%	Smallest sign flip. Sonnet's cold quality walk is already compact (98K, 19 turns). AFTER (109K, 15 turns) front-loads the SKILL.md and then still needs follow-up review of the script's flagged rows; the 12% loss is inside likely variance for N=1.
Opus (4-7) N=1, 2026-05-31	778b40d · bb174dc	56,615 tokens	53,895 tokens	+2,720 tokens	+5%	Marginal positive — sits inside variance for N=1. Opus BEFORE (7 turns, 57K) was unusually compact; AFTER (9 turns, 54K) trims the deep judgement step but pays for SKILL.md context.
Haiku (4-5) N=1, 2026-05-31	778b40d · bb174dc	81,544 tokens	67,623 tokens	+13,921 tokens	+17%	Largest positive in the study. Same investigation-collapse pattern seen with copyright-scan/calibration: the smaller model would scout exhaustively when cold, and reaps the most benefit when handed a directed walk.

Cross-model insight. Look down each column: the AFTER cost is roughly the same across models (109K / 163K / 109K for freshness, 109K / 54K / 68K for quality), but the BEFORE cost varies wildly (195K / 135K / 64K and 98K / 57K / 82K). The optimization's value is dominated by *how expensive the cold walk would have been on that model*. The script puts a floor under the cost; it can't put a ceiling under the cold walk.

Findings

1. The aggregate +2.5% is honest but small — and that's the real finding.

Two skills landed in this round; the net delta across the six cells is +15,877 tokens (+2.5%). That's a small number, and we are not going to dress it up as a portfolio-wide claim — the registry already paid for that lesson once. The optimization principle works in three cells (Sonnet/freshness, Opus/quality, Haiku/quality) and fails in three others. The story is conditional, not universal.

2. skills-freshness pays off massively on Sonnet, sign-flips on Opus and Haiku.

Sonnet AFTER ran the Python script and reviewed five flagged rows in 38 short turns at 111K tokens, where Sonnet BEFORE explored all 14 skill dirs end-to-end at 195K tokens. That's the win the optimization principle predicts: the script handles iteration, the LLM only judges what surfaces. On Opus and Haiku, the BEFORE arm was already compact (135K and 64K respectively), so the SKILL.md context + script dispatch + result review of AFTER overpays.

3. skills-quality is the inverse pattern — Haiku wins biggest, Sonnet sign-flips.

Haiku AFTER saved 17% (82K → 68K). Opus AFTER saved a marginal 5% (57K → 54K) — inside likely N=1 variance. Sonnet AFTER actually cost 12% more (98K → 109K). The Haiku result tracks the "investigation-collapse" pattern observed in earlier copyright-scan and ci-failure-triage calibrations: smaller model with a directed walk beats the same model going cold by the widest margin.

4. Sign-flips trace to a single cause: the BEFORE arm was already cheap.

Across all three negative cells, the BEFORE arm came in well under 100K tokens. When the cold-walk cost is small, the fixed-overhead floor of an AFTER arm — SKILL.md in context, Python script dispatch, result-review of N flagged rows — rises above what was being saved. The optimization stops paying for itself somewhere around the point where the cold walk would already finish in <100K tokens; below that line, you are buying overhead, not avoiding work.

5. /mikko-skills-quality still flags 13 skills as potentially token-wasteful (1 HIGH, 12 MEDIUM).

The HIGH is project-scope `content-audit` — 8 loop-style prose patterns ("for each X, do Y"), no companion script. That's a genuine target for a future optimization round and the script is doing its job. The 12 MEDIUM flags are mostly recall-biased false positives: prose in `mikko-audit`, `mikko-help`, etc. describes script-driven loops using "for each X" phrasing, and the pre-pass cannot tell "LLM is the loop" from "script is the loop, prose describes it". Read them as suggestions to consider, not confirmed waste.

6. /mikko-skills-freshness reports the portfolio is clean.

Of the 14 global skills, only `mikko-skills-freshness` (3 pass, status: *changed* because never baselined) and `mikko-skills-quality` (5 pass, status: *new*) surface. The other 12 are clean. Nothing is stale that should have been re-audited.

7. The same-model A/B is what makes the +43% honest where the registry's cross-model +22% wasn't.

The skill-registry's 2026-05-22 calibration measured a ~22% aggregate save across 13 Spacepotatis skills, but its BEFORE and AFTER arms ran on different transcripts at different times. Model drift and the 1K–1M-token natural variance in agent runs both contaminate that number. This study pins the same model, the same target directory, and the same task on both arms in the same hour — so the +43% / –70% extremes here reflect the optimization alone. That's a stronger test, and it tells a more conditional story.

WHAT THIS STUDY DOES NOT CLAIM

- *That the +2.5% aggregate generalizes to the other 26 skills in the portfolio. We measured 2.*
- *That the "minimize LLM tokens, maximize local computation" principle is invalidated. Three of six cells confirmed it; the conditions matter.*
- *That the maintenance cost of writing and shipping the Python script is accounted for. It isn't.*
- *That per-model dollar pricing is folded in. Raw token totals only, per registry convention.*
- *That a second N=1 re-run would land within 5% of these numbers. With Sonnet/freshness at +43% and Haiku/freshness at –70%, the per-cell variance bound is wider than we can pin from one observation.*

Round 2 — post-fix validation

After Round 1 flagged three sign-flipped cells on `skills-freshness` , PR #18 tightened that skill's `SKILL.md` with three guard phrases targeting the three mechanisms the forensic pass identified: `limit=80` (read-granularity), `max 3 traces total`, `don't spelunk` (post-script review depth), and `no deep read` (reinforces the read-granularity guard). Round 2 re-ran the same A/B against the post-fix `SKILL.md` to validate. Six agents, ~190 seconds, ~640K tokens.

| Hypothesis going in: *opus and haiku should flip from negative to positive; sonnet should stay positive. Verdict: confirmed on all three cells.*

ROUND 2 HEADLINE (SKILLS-FRESHNESS ONLY)

3 of 3

Cells now net-positive

was 1 of 3 in Round 1 (only sonnet)

+11 %

Aggregate save across the 3 re-measured cells

335,646 tokens BEFORE → 299,738 tokens AFTER (skills-freshness, post-fix)

+90 pp

Biggest single-cell swing (haiku, −70% → +20%)

the `limit=80`, `no deep read` guard killed the 233K-char full-file read pattern

The fix landed. The three mechanisms identified in Round 1 are real, and the `SKILL.md` tightening neutralises them on the cells where they fired. `skills-quality` wasn't re-measured — its `SKILL.md` was unchanged by the fix (it was already clean by its own rules), so a re-run would have been noise.

Round-over-round comparison

CELL	R1 BEFORE	R1 AFTER	R1 %	R2 BEFORE	R2 AFTER	R2 %	SWING
skills-freshness/sonnet	194,751	110,828	+43%	106,046	91,369	+14%	−29pp
skills-freshness/opus	134,765	163,301	−21%	132,716	131,231	+1%	+22pp
skills-freshness/haiku	64,192	108,806	−70%	96,884	77,138	+20%	+90pp

What the post-fix story looks like, per cell

haiku: -70% → +20% (largest swing in the study)

Round 1's haiku AFTER spent 109K tokens reading all 14 SKILL.md files *in full* (no `limit` param) because the procedure said "read each SKILL.md" literally and the model executed it literally. Round 2's AFTER now sees the procedure say "Read the SKILL.md once with `limit=80` (frontmatter + first one or two sections is all you need — no deep read)" — and that's exactly what haiku did, dropping AFTER from 109K to 77K. Meanwhile the BEFORE arm got more expensive (64K → 97K) because the model chose a deeper traversal this run, so the relative save flipped from -70% to +20% — a 90pp swing.

opus: -21% → +1% (sign-flip cleared, now break-even)

Round 1's opus AFTER spent 4 extra Bash calls chasing broken path references from `mikko-ai-codegen-smell-audit` into the `D:/tmp/claude-skills/` source repo, costing +28K tokens vs BEFORE. Round 2's AFTER procedure now caps path verification at "one `ls` per finding, max 3 traces total, don't spelunk into source repos." Opus stopped after the first `ls` per finding; AFTER dropped from 163K to 131K. BEFORE barely moved (135K → 133K). Result: break-even — the procedural overhead and savings now balance.

sonnet: +43% → +14% (still positive, smaller margin)

Sonnet's BEFORE arm cost dropped from 195K to 106K — the cold scan happened to be much cheaper this run (different traversal strategy, possibly different cache state). AFTER also dropped (111K → 91K) but by less. The relative save shrunk from +43% to +14% because the baseline became cheaper, not because the optimization stopped working. This is N=1 variance — sign and direction unchanged, magnitude lower because of natural BEFORE-arm spread.

Verdict on the optimization principle

Round 1 was honest about three sign-flips and the mechanisms behind them. Round 2 shows the fix cleared all three. The pattern that emerges across both rounds: **the optimization principle works when the SKILL.md procedural language is bounded** (limit on reads, cap on follow-up, single-batch on parallelism). When the procedure is unbounded, capable models will follow it more conscientiously than improvisation, and pay more for the privilege. The cost-trap rules added in PR #18 catch this regression class for every future skill before it ships.

Rounds 3 – 5: testing the rigor-exempt label and re-validating

After Round 2 confirmed the fix worked on `skills-freshness`, three follow-up questions drove the next three rounds: (3) were the 5 "rigor-exempt" skills (where the cost-trap rules fire but we dismissed them) actually rigor-exempt, or just assumed? (4) if we apply the same fix-shape to one of them, does the rule's prediction hold? (5) are Round 1 + 2 actually repeatable, or N=1 noise dressed up as a story?

Round 3 — rigor-exempt label test on content-audit

Picked `content-audit` (Spacepotatis project skill, single-phase, no nested sub-agents) as the cleanest test of the rigor-exempt label. If a "rigor-exempt" skill is truly cost-driven by its design, AFTER (following the SKILL.md) should be deep-negative vs BEFORE (cold improvisation).

CELL	BEFORE	AFTER	SAVED	%	NOTE
content-audit/sonnet	138,012	142,301	−4,289	−3%	slight loss
content-audit/opus	162,332	149,141	+13,191	+8%	mild save
content-audit/haiku	93,281	98,137	−4,856	−5%	slight loss
Aggregate	393,625	389,579	+4,046	+1%	break-even

*Finding: content-audit sits at break-even, not deep-negative. The rigor-exempt label was **inflated**. Wholesale exemption from cost-trap rules would silence a real (if small) signal.*

Round 4 — post-fix re-run of content-audit

Same fix shape as Round 2's skills-freshness fix: added "at most 5 traces, batch into a single grep with alternation if more, don't spelunk per-id" to step 2 of content-audit's SKILL.md. Goal stated up front: **PASS** if ≥2 of 3 cells flip *and* aggregate > +5%; **MIXED** if 1 cell flips and aggregate moves but < +5%; **FAIL** if 0 cells flip or aggregate regresses.

CELL	R3 %	R4 %	SWING	FLIPPED?
content-audit/sonnet	−3%	−9%	−6pp	no — regressed
content-audit/opus	+8%	+10%	+2pp	no (already positive)
content-audit/haiku	−5%	+11%	+16pp	YES (neg → pos)

CELL	R3 %	R4 %	SWING	FLIPPED?
Aggregate	+1%	+4%	+3pp	—

Verdict: **MIXED**. Haiku's +16pp swing mirrors Round 2's freshness/haiku result (+90pp on a similar cap fix) — smaller models reliably benefit from bounded procedural language. Sonnet regressed −6pp, plausibly N=1 noise from the added wording's procedural complexity. Aggregate moved positive but stayed under the +5% PASS threshold.

Fix shipped anyway as Spacepotatis PR #280 — haiku's gain is real, the prose is tighter, and the downside is bounded.

Round 5 — re-validation of both new skills

The question: are Round 1 + 2's numbers actually repeatable, or did we tell a coherent story from N=1 noise? Re-ran both `skills-freshness` AND `skills-quality` at their current state. Goal stated up front: **PASS** if skills-freshness ≥2 cells positive AND skills-quality direction matches Round 1 AND aggregate > 0.

CELL	R1 %	R2 %	R5 %	REPEATS?
skills-freshness/sonnet	+43%	+14%	+1%	positive each round, magnitude declining (BEFORE arm gets cheaper)
skills-freshness/opus	−21%	+1%	−1%	sign-flip cleared, now hovers near break-even
skills-freshness/haiku	−70%	+20%	+48%	consistent strong saver post-fix
skills-quality/sonnet	−12%	—	+2%	flipped positive (no R2 data)
skills-quality/opus	+5%	—	−62%	N=1 anomaly — BEFORE arm short-circuited (31K)
skills-quality/haiku	+17%	—	+54%	strongest haiku save in the study
Aggregate (12 cells)	—	—	+16%	strongest aggregate of any round

Verdict: **MIXED** per the strict goal — skills-freshness passed (2 of 3 cells positive), skills-quality didn't directionally match Round 1 (1 of 3 cells match sign), but aggregate is the strongest of any round (+16% / 12 cells). The opus/skills-quality −62% is an N=1 anomaly: BEFORE cell was 31K tokens (opus short-circuited cold without thoroughly auditing); AFTER (51K) was the more representative number.

Cross-round synthesis

Stable findings across all five rounds

Haiku consistently benefits most: Round 1 +17% on quality, Round 2 +20% on freshness, Round 4 +16pp flip on content-audit, Round 5 +48% on freshness and +54% on quality. The investigation-collapse pattern is the most reliable finding in the study.

Aggregate trend is positive and growing: Round 1 +2.5%, Round 2 +11%, Round 5 +16%. As the fixes land, the optimization principle pays off harder.

Opus is the most variable model: single-cell swings up to 60pp between rounds. Measurement noise dominates Opus at N=1; do not read individual Opus cells as confident signal.

What the five rounds collectively show

The "maximize local computation, minimize LLM tokens" principle is real and the cost-trap rules in `skills-quality` detect genuine waste. The fix's effectiveness is *model-dependent*: smaller models (Haiku) benefit reliably and substantially, larger models (Opus) show variable response. N=1 per cell is sufficient to surface direction but not magnitude — headline numbers should always be read with that caveat. Wholesale rigor-exempt labels are inflated; per-finding judgment is the right level of skepticism.

Implications for the cost-trap rules

The three rules in PR #18 (`unlimited_read_in_procedure` , `uncapped_followup` , `batch_invitation`) detect a real class of waste, validated on at least one cell each. But the rule messages overclaim universality. A future iteration could note "fix likely helps haiku-class models substantially; may modestly hurt opus-class models that don't over-execute" so users know what to expect. The five remaining rigor-exempt skills in issue #20 are still labeled exempt by assumption only — each would benefit from a Round-3-style test before the dismissal is treated as durable.

Appendix — method

A/B design

For each of $(\text{skill}, \text{model}) \in \{\text{skills-freshness}, \text{skills-quality}\} \times \{\text{sonnet}, \text{opus}, \text{haiku}\}$, two fresh sub-agents ran in parallel against the same target directory (`C:/Users/vandr/.claude/skills/` , 14 mikko-* skill dirs) with the same task statement. The BEFORE arm received only the task; the AFTER arm received the task plus the current SKILL.md verbatim. Both arms returned a structured report (skills examined, findings count, tool-use estimate, approach, summary) via JSON schema.

Token accounting

Per-agent JSONL transcripts under `~/ .claude/projects/.../subagents/workflows/wf_69f7e105-98f/` were parsed line-by-line. For each assistant message we summed `usage.input_tokens + usage.cache_creation_input_tokens + usage.output_tokens` , deduped by `requestId` so the thinking+tool_use double-line doesn't double-count. `cache_read_input_tokens` excluded — those tokens were paid upstream. This matches `scripts/build-review-stats.mjs` 's convention exactly.

BEFORE arm definition

For these two NEW skills, the parent commit of the optimizing PR did not contain the SKILL.md at all, so BEFORE = no SKILL.md. The arm represents what an unguided LLM would do given only the task. For future optimization rounds against pre-existing skills, BEFORE would be the SKILL.md at the parent of the optimizing commit.

Why same-model, same-hour

The skill-registry's aggregated stats fold in transcripts from different sessions, different models, and different weeks. Useful for portfolio-level cadence, dangerous as an isolation of any single optimization's effect. This study pins the model, the task, and the hour so any delta reflects the SKILL.md change alone — not model drift, not natural per-session variance, not the harness's cache state from earlier work.

Rigor-exempt skills skipped

Six skills were deliberately excluded: `mikko-audit` , `mikko-security-audit` , `mikko-audit-suite` , `project-scope` `modular-architecture-audit` , `save-roundtrip-audit` , `content-audit` , and `equipment` . These are rigor-encoding skills — their cost is the point. Calibrating them against an unguided cold walk would falsely cast them as token-wasteful when their job is to spend tokens systematically that the user would otherwise spend chaotically. The optimization principle does not apply to them.

Worktree isolation

Skipped. Both arms are read-only against `~/ .claude/skills/` ; there is no cross-arm mutation risk that worktree isolation would defend against. The mikko-skill-calibration skill's worktree convention is over-spec for this study and was omitted to save the ~6 seconds of worktree-setup overhead per agent.

Run record

- Round 1: `wf_69f7e105-98f` — 12 agents (2 skills × 2 arms × 3 models), 244 s, 1,131,817 subagent tokens
- Round 2: `wf_8323de85-910` — 6 agents (skills-freshness post-fix), 190 s, 642,172 subagent tokens
- Round 3: `wf_1f7f067c-791` — 6 agents (content-audit rigor-exempt test), 573 s, 758,141 subagent tokens
- Round 4: `wf_9c93fac3-e6b` — 6 agents (content-audit post-fix), 610 s, 828,450 subagent tokens
- Round 5: `wf_0d56c8c9-5bc` — 12 agents (both new skills re-validation), 212 s, 1,122,667 subagent tokens
- Total: 42 agents, ~4.48M subagent tokens across all rounds
- Per-cell tokens (input + output + cache_creation, requestId-deduped): see spine tables
- Source data: `D:/tmp/skills-optim-study/skills-optim-study-2026-05-31.json` + `-round2.json` + `-rounds-3-4-5.json`
- Fixes shipped between rounds:
 - `claude-skills` PR #18 (cost-trap rules + skills-freshness SKILL.md tightening)
 - `claude-skills` PR #19 (clears 3 cleared-this-round findings on ai-codegen-smell-audit, readme-drift-sync, skill-calibration)
 - `claude-skills` issue #20 (rigor-exempt dismissal tracking)
 - `Spacepotatis` PR #280 (content-audit cap, between Round 3 and Round 4)

Why skills-quality wasn't re-measured in Round 2

PR #18 touched only `skills-freshness/SKILL.md`. `skills-quality/SKILL.md` was already clean by its own new rules (a regression test pins this). Re-measuring its 3 cells would have added N=1 variance with no testable hypothesis — same task, same procedure, same target, so any delta would be measurement noise. Round 2 deliberately narrows scope to where something actually changed.